CreditRisk Interpretation RAG System: Leakage-Aware Default-Risk Modeling for LendingClub Loans

Linda Perez Penaranda Yashaswi Aryan Siddharth Agarwal

Abstract

This project develops a leakage-aware credit-risk interpretation prototype for LendingClub accepted loans. The system defines a strict completed-loan charge-off target, removes post-origination leakage fields, uses chronological validation, compares major tabular model families, calibrates probabilities, interprets the selected model with SHAP, and generates adverse-action-style explanation drafts using retrieval over U.S. regulatory sources. From 2,260,701 accepted-loan records, the final completed modeling frame contains 1,345,310 loans and 38 starter features. The preferred neutral XGBoost model without LendingClub grade/subgrade achieves test ROC-AUC 0.710, PR-AUC 0.381, and a top-20% bad-rate concentration near 40%. A retrieval-augmented generation (RAG) prototype improves blind statutory-accuracy score from 0.60 to 0.76 versus a no-RAG control on a small five-letter-per-mode evaluation. The work is best understood as a research-grade risk-ranking, interpretation, and explanation-grounding workflow prototype of a production lending system or a compliance-approved adverse-action notice engine.

1 Introduction

Consumer credit underwriting requires ranking default risk while giving defensible, specific explanations for adverse decisions. LendingClub's public dataset supports this problem because the accepted-loan file contains loan, borrower, pricing, and credit-bureau-style variables together with eventual repayment outcomes. The supervised task in this project is binary classification of completed accepted loans: `target_bad=0` for `Fully Paid` and `target_bad=1` for `Charged Off`.

The dataset also creates common credit-modeling hazards. Defaults are a minority class, so accuracy is not an adequate objective. LendingClub vintages span changing borrower mix and macroeconomic conditions, so random splits can overstate future performance. Many raw fields are observed after origination, including payment, recovery, hardship, settlement, and last-credit-pull information; using them would leak the target. Rejected applications are excluded from supervised default modeling because they have no observed repayment outcomes in this dataset.

The final workflow addresses these risks with auditable cleaning, leakage screening, chronological splits, minority-class metrics, calibrated probabilities, SHAP interpretation, and a capped economic policy. It also implements a RAG explanation layer: SHAP selects borrower-level reasons, a hybrid retriever supplies regulatory context, and a writer model drafts constrained adverse-action-style letters for comparison with a no-RAG baseline.

2 Proposed Method Summary

The proposed method is an end-to-end accepted-loan default-risk and explanation-grounding pipeline. It estimates $P(\texttt{target\_bad} = 1)$ using cleaned application-time features, selects a governed model after ablation and calibration review, explains model behavior with SHAP, and grounds generated explanation drafts

in retrieved ECOA, Regulation B, FCRA, and CFPB materials. This approach is preferable to optimizing F1 alone because the best-F1 threshold flags 42.07% of test loans, an unrealistic action volume for underwriting. It is also preferable to relying directly on LendingClub grade/subgrade because those fields summarize LendingClub's prior risk decisioning. The selected neutral XGBoost model removes explicit grade and subgrade while retaining `int_rate_clean`, preserving ranking quality with a cleaner governance story.

3 Related Work

Prior LendingClub credit-risk studies motivate both the predictive and interpretability components of this project. Gupta, Gulati, and Chakrabarty use the same Kaggle LendingClub dataset with exploratory analysis, Logistic Regression, Random Forest, and expected-loss concepts such as probability of default, loss given default, and exposure at default [5]. Their study supports the financial-risk framing, while this project strengthens the workflow with explicit leakage audits, chronological validation, calibration, and operating-policy constraints.

Explainability papers show why predictive scores alone are insufficient in credit-risk settings. Hadji Misheva et al. apply explainable AI methods, including SHAP-style explanations, to credit-risk management and emphasize that explanations should support model governance rather than merely decorate predictions [6]. Demajo, Vella, and Dingli similarly study interpretable credit scoring with XGBoost and explanation methods on LendingClub and HELOC-style data [7]. This project follows that separation by using SHAP to explain model behavior, then treating applicant-facing letters as a separate retrieval-grounded generation problem.

Recent language-model and text-feature work shows a possible but risky extension path. Sanz-Guerrero and Arroyo use borrower-written loan descriptions with BERT-derived indicators in peer-to-peer lending [8]. Their result suggests that text can contain risk signal, but this project intentionally avoids borrower free text in the main model because it can introduce privacy, proxy-variable, and applicant-facing explanation risks.

Feature-governance work is directly relevant to the selected model. Schwartz, Wang, and Fang use SHAP-guided feature reduction to improve interpretability in credit scoring [9]. This supports our grade/subgrade ablation: the selected model sacrifices little ranking performance while reducing direct dependence on LendingClub's opaque internal risk buckets.

More complex ensemble and monitoring studies suggest future improvements but also explain why this project remains conservative. Nortey et al. report strong loan-default results with optimized greedy weighted ensembles [10], while Solozobov studies label-free evidence degradation monitoring for credit-risk systems when outcomes arrive late [11]. Those directions are valuable, but the present method is better aligned with the course objective because it prioritizes reproducible validation, leakage control, calibration, SHAP interpretation, and RAG evidence grounding over maximum model complexity.

4 Related Implementations

Nathan George's Kaggle Python notebook for the LendingClub dataset demonstrates practical loading, basic status exploration, and starter EDA for the accepted-loan file [2]. That implementation is useful for understanding the raw data shape, but this project goes further by defining a strict terminal-outcome target, excluding unresolved loans, documenting post-origination leakage, and evaluating on chronological holdout periods.

The companion Kaggle R notebook provides another implementation path for initial LendingClub exploration and reinforces the value of simple summary tables before modeling [3]. Our method is stronger for

the final task because it moves from descriptive EDA to a full supervised pipeline: cleaning, preprocessing, model-family comparison, calibration, ablation, SHAP interpretation, and policy analysis.

Gusarova's Kaggle notebook focuses on detecting and removing outliers in the same LendingClub data [4]. This project uses a more conservative credit-risk approach: suspicious values are not automatically deleted unless a documented business or leakage rule justifies the exclusion, because aggressive outlier removal can remove legitimate high-risk borrowers and distort default-risk evaluation.

5 Dataset and Inputs

The raw source is the Kaggle LendingClub release containing accepted and rejected files from 2007–2018Q4 [1]. The accepted file used here has 2,260,701 rows and 151 columns. The supervised model estimates default risk conditional on historical LendingClub approval; it does not estimate the risk of rejected applicants without reject inference or external outcome data.

The final strict target maps `Fully Paid` to 0 and `Charged Off` to 1. Unresolved or snapshot statuses such as `Current`, `In Grace Period`, late statuses, and policy-exception statuses are excluded because their terminal outcomes are not stable labels. The resulting completed modeling frame contains 1,345,310 rows: 1,076,751 fully paid loans, 268,559 charged-off loans, and an observed bad rate of 19.96%.

Cleaning preserves raw fields and creates auditable helper variables. Percentage strings become numeric `int_rate_clean` and `revol_util_clean`; `term` becomes `term_months`; employment length is normalized; dates are parsed; FICO endpoints are averaged into `fico_mean`; and credit-history age is computed from issue date and earliest credit-line date. Identifiers, target fields, payment totals, outstanding principal, recoveries, collection fees, hardship fields, settlement fields, last-payment fields, and last-credit-pull fields are removed from the modeling matrix.

6 Data Analysis

EDA first established the correct modeling population. The accepted-loan file contains terminal outcomes, active accounts, late statuses, and other nonterminal states. The analysis confirmed that current loans should not be treated as good loans, because doing so would label censored active accounts as non-defaults and bias the target toward lower observed default risk.

The target analysis therefore used the strict completed-loan mapping: `Fully Paid` equals 0 and `Charged Off` equals 1. This produced 1,345,310 completed modeling rows and a 19.96% observed bad rate. That base rate makes accuracy a weak metric, because a classifier can appear strong by overpredicting the majority fully paid class.

Feature analysis showed intuitive credit-risk gradients. Higher grade/subgrade risk, higher interest rate, longer term, higher DTI, higher revolving utilization, larger loan amount, and more recent account openings are associated with higher observed bad rates. Higher FICO and higher income are associated with lower risk, although income is skewed and must be interpreted cautiously.

Several transformations made raw LendingClub fields usable without losing traceability. Percentage strings were parsed into numeric `int_rate_clean` and `revol_util_clean`; `term` was converted to `term_months`; and employment length was normalized. The two most important derived formulas were

$$\text{fico\_mean} = \frac{\text{fico\_range\_low} + \text{fico\_range\_high}}{2}, \quad \text{credit\_history\_years} = \frac{\text{issue\_date} - \text{earliest\_credit\_line}}{365.25}.$$

Missingness was handled as a modeling issue rather than as an automatic deletion rule. Some credit variables are structurally missing, such as months since last public record when no public record exists. The

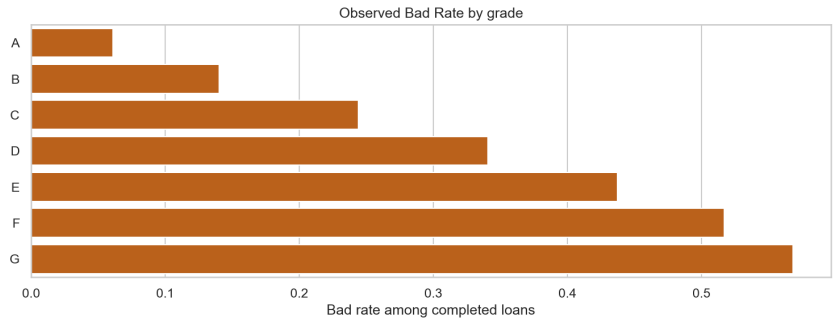

Figure 1: Observed bad rate by LendingClub grade. Grade is useful for EDA but removed from the preferred model.

analysis therefore distinguished structural missingness from ordinary low-missingness numeric fields and deferred imputation until after train/validation/test splitting.

Temporal analysis motivated chronological validation. LendingClub volume, grade mix, interest rates, and default rates vary across issue years. The final split trains on older loans, validates on the next period, and evaluates once on the newest holdout, which better approximates future lending than a random split.

7 Methods

Let $y \in \{0, 1\}$ denote charge-off status for a completed accepted loan. Each model estimates a score $\hat{p}(x) = P(y = 1 \mid x)$. Logistic Regression provides a transparent linear-logit baseline:

$$\log \frac{\hat{p}(x)}{1 - \hat{p}(x)} = \beta_0 + x^\top \beta.$$

It is useful because coefficients are easy to inspect, but it can miss nonlinear interactions among income, loan amount, term, utilization, and credit history.

Tree ensembles were used because the dataset is tabular, heterogeneous, and interaction-heavy. Random Forest averages many decorrelated trees, reducing variance relative to a single decision tree. HistGradientBoosting, LightGBM, XGBoost, and CatBoost build additive trees sequentially, where each new tree improves errors left by earlier trees; these models are well suited to threshold effects and borrower-feature interactions.

The project evaluates discrimination and minority-class detection rather than accuracy. Precision is the share of flagged loans that charged off; recall is the share of charge-offs captured; and

$$F_1 = 2 \cdot \frac{\text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}.$$

ROC-AUC summarizes threshold-free ranking, while PR-AUC is emphasized because charge-offs are the minority class. Thresholds were selected on validation data, not on the test set, so the final test metrics represent a future-period holdout.

Preprocessing and tuning were deliberately conservative. Categorical variables were encoded after splitting, numeric imputation was fit on training data, and model-family candidates were compared using validation PR-AUC, ROC-AUC, and F1 rather than accuracy. The project also ran an advanced PR-AUC-oriented search for gradient-boosting candidates, then used final comparison and ablation outputs to choose the preferred model.

Calibration and thresholding translate scores into decisions. Platt sigmoid calibration supports probability-based policy analysis. The economic policy uses

$$\text{EV} = TP \cdot L_{\text{saved}} - FP \cdot C_{\text{opportunity}},$$

with an action-share cap; the example assumptions are $10,000 saved per caught default, $1,500 opportunity cost per rejected good borrower, a 20% maximum action share, and a 13.04% break-even default probability.

SHAP explains the selected XGBoost model globally and locally. The interpretation notebook computes global mean absolute SHAP values, dependence plots, waterfall examples, and validation-versus-test stability on reproducible samples. The generation layer then uses deterministic SHAP reason selection: high-risk examples only, risk-increasing drivers only, mapped features only, `int_rate_clean` excluded from applicant-facing reasons, and at most four principal reasons.

The RAG corpus contains eight public regulatory sources: Regulation B definitions and notice requirements, Regulation B Appendices A and C, official commentary to section 1002.9, FCRA section 615, and CFPB Circulars 2022-03 and 2023-03. The chunker strips boilerplate, preserves citation metadata, and produces 70 chunks. Retrieval combines MiniLM dense embeddings, BM25 lexical search, and reciprocal-rank fusion. RAG generation uses fixed legal-envelope queries and a constrained writer prompt; the no-RAG control uses the same SHAP reasons without retrieved law. A blind shuffled judge scores statutory accuracy on five binary items.

8 Experimental Setup

The chronological split contains 962,641 train rows from 2007-06-01 to 2016-04-01 with 18.83% bad rate, 186,920 validation rows from 2016-05-01 to 2017-02-01 with 24.68% bad rate, and 195,749 test rows from 2017-03-01 to 2018-12-01 with 21.03% bad rate. Preprocessing creates baseline and no-grade/subgrade feature sets, applies split-consistent imputation and encoding, and exports model matrices and manifests.

The project repository is lindaperez/CreditRiskRAG. The notebooks containing the experimental setup are public project artifacts on the repository's development branch: EDA, cleaning, preprocessing, final model comparison, and SHAP interpretation. These notebooks define the target, leakage exclusions, split rule, feature matrices, model selection, calibration review, and interpretation outputs.

Model notebooks under `Modeling/` cover Logistic Regression, Random Forest, HistGradientBoosting, LightGBM, XGBoost, CatBoost, advanced PR-AUC candidate search, grade/interest-rate ablation, and final comparison. Reproducibility artifacts include CSV outputs for split summaries, candidate metrics, selected metrics, calibration, economic policy, SHAP interpretation, and RAG evaluation. The environment is documented in `environment.yml` and `requirements.lock.txt`; SHAP interpretation uses `shap==0.51.0`.

9 Results

Table 1 summarizes the main model comparison at validation-selected F1 thresholds. Gradient boosting methods perform similarly. LightGBM is the best technical F1 benchmark, while the preferred neutral XGBoost no-grade/subgrade model is selected for governance, calibration, SHAP compatibility, and nearly identical ranking performance.

The preferred model reaches validation ROC-AUC 0.702, PR-AUC 0.426, precision 0.361, recall 0.696, and F1 0.475. On the future-period test set it reaches ROC-AUC 0.710, PR-AUC 0.381, precision 0.330, recall 0.660, and F1 0.440. Its mean raw predicted probability on test is 0.197, close to the actual test bad rate of 0.210, which is more usable than earlier class-weighted variants with inflated probabilities.

The ablation shows that grade/subgrade and interest rate carry overlapping information. Removing explicit grade/subgrade while retaining `int_rate_clean` slightly improves validation PR-AUC and top-20% test bad-rate concentration relative to keeping both. Removing both grade/subgrade and interest rate causes the clearest degradation.

The best-F1 threshold is not the final business recommendation because it flags 42.07% of test loans. The capped economic policy flags 19.47% of test loans, captures 37.43% of defaults, produces a 40.44% bad rate

Table 1: Model comparison at selected validation threshold.

Model	Val PR	Val F1	Test ROC	Test PR	Test P	Test R
Logistic Regression	.412	.471	.700	.362	.311	.709
Random Forest	.415	.468	.704	.371	.321	.674
HistGradientBoosting	.424	.474	.708	.377	.320	.693
LightGBM	.425	.475	.711	.381	.316	.718
XGBoost	.425	.474	.710	.380	.319	.701
CatBoost	.421	.472	.706	.374	.317	.697
Preferred neutral XGB	.426	.475	.710	.381	.330	.660

Table 2: Grade/subgrade and interest-rate ablation.

Feature policy	Val PR	Test PR	Test ROC	Test top-20% bad
Grade/subgrade + interest	.426323	.380704	.710495	39.98%
No grade/subgrade + interest	.426423	.380806	.710201	40.09%
Grade/subgrade, no interest	.426470	.381304	.710762	39.98%
No grade/subgrade, no interest	.422215	.375358	.703070	39.64%

in the action group, leaves a 16.34% bad rate among approved loans, and has estimated value of $120.0M under the stated example assumptions. This is a more defensible operating policy than an unconstrained metric optimum.

SHAP confirms plausible model behavior. The top global drivers are `int_rate_clean`, `term_months`, `acc_open_past_24mths`, `dti`, `fico_mean`, `annual_inc`, and `loan_amnt`. Validation-versus-test SHAP rank stability has Spearman correlation 0.9987, and the top six features have no rank shift, supporting explanation stability across the holdout period.

The RAG layer is implemented as a small but informative prototype. It generated five RAG and five no-RAG letters from the same SHAP-selected principal reasons. Both modes preserved selected reasons, the FCRA 60-day free-report window, and ECOA protected-class language. RAG improved the overall blind statutory-accuracy score from 0.60 to 0.76, mainly by improving real federal-agency naming and reducing legal-error failures from 0.00 to 0.40. The remaining failures show that retrieval helps but does not guarantee compliance; deterministic notice blocks and larger evaluation samples are needed.

10 Analysis and Discussion

The model works because consumer-credit risk is nonlinear and interaction-heavy. Interest rate, term, installment burden, DTI, income, FICO, recent account openings, and loan amount do not act independently. Tree ensembles can model these interactions while still allowing post hoc explanation through SHAP.

The selected feature policy is a governance compromise. LendingClub grade and subgrade are predictive but encode the platform's prior underwriting judgment. Retaining the continuous interest-rate signal while removing discrete grade buckets keeps nearly all ranking performance and simplifies explanation review, although interest rate itself remains policy-sensitive because it reflects pricing.

Temporal drift remains a central limitation. The validation bad rate is 24.68%, while the test bad rate is 21.03%. This reinforces the need for calibration review, threshold monitoring, and future-period validation before deployment. The accepted-loan boundary is equally important: the model ranks default risk among loans LendingClub historically approved, not among all applicants.

The solution generalizes best to lending or account-risk datasets with delayed outcomes, time-varying

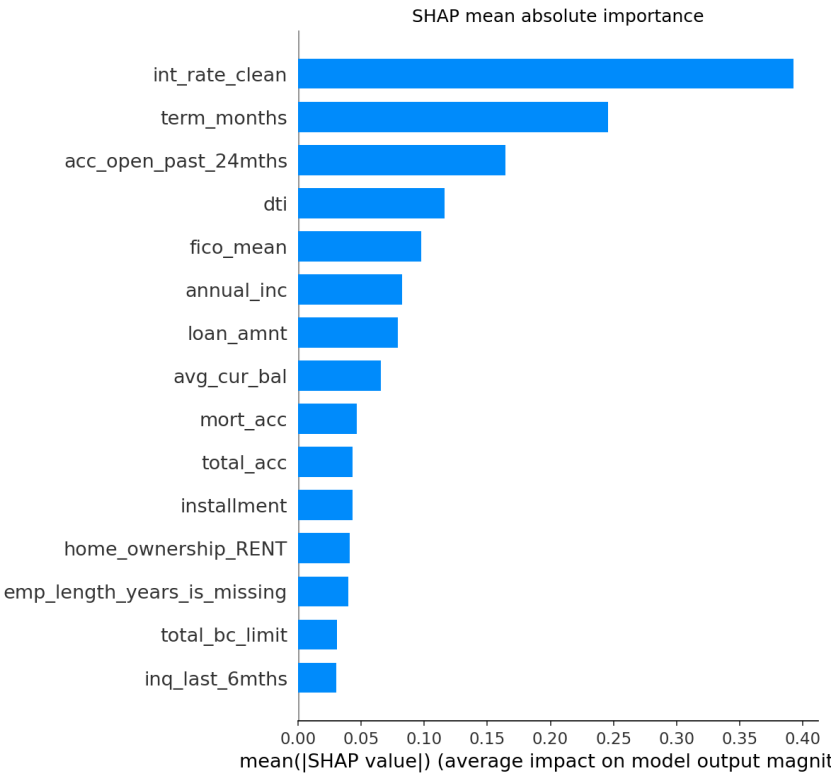

Figure 2: Global SHAP mean absolute importance for the preferred XGBoost model.

portfolios, mixed numeric and categorical features, and strong leakage risk from post-decision fields. It should transfer less directly to rejected-applicant scoring, thin-file credit populations, or products with materially different loss definitions unless new labels, fairness review, and calibration data are available.

The explanation layer should be described positively but conservatively. The implemented corpus, hybrid retriever, deterministic reason selector, RAG generator, no-RAG control, and blind judge create a credible research experiment. However, five letters per mode are not sufficient for compliance claims, and agency-name errors show why adverse-action notices require rule-based required-field validation and human compliance review.

11 Conclusion

This project built a leakage-aware LendingClub default-risk modeling and explanation-grounding workflow from raw data through EDA, cleaning, preprocessing, model comparison, ablation, calibration, SHAP interpretation, economic policy, regulatory retrieval, letter generation, and blind RAG/no-RAG evaluation. The preferred neutral XGBoost model without grade/subgrade achieves stable future-period ranking performance, and SHAP identifies intuitive, stable risk drivers.

For a company considering this workflow, the recommended configuration is the calibrated neutral XGBoost model with the capped economic review policy, not the unconstrained best-F1 threshold. This choice better balances risk concentration, review volume, probability calibration, and governance than a purely metric-optimized decision rule.

With more data and time, the next work should add external validation, reject-inference research, fairness testing, deterministic adverse-action notice templates, larger RAG/no-RAG evaluation samples, and

monitoring for temporal drift. Until those controls are complete, the model should be used as a research-grade accepted-loan risk-ranking and explanation prototype rather than a fully automated lending-decision system.

12 AI Prompts Used

AI tools were used for research planning, coding support, debugging, explanation drafting, report drafting, RAG letter generation, no-RAG control generation, and LLM-based evaluation. The recoverable prompt inventory is documented in `Docs/Prompts/AI_Prompts_Used.md` and summarized in `Docs/Prompts/Prompt_Tree_And_Inventory.md`. The saved prompts include the full EDA notebook prompt, the final-report LaTeX drafting prompt, three legal retrieval queries, the RAG adverse-action-style writer prompt, the no-RAG control prompt, and the compliance judge rubric. Any additional Claude, ChatGPT, or other conversational prompts used by individual team members must be pasted from their chat histories into the prompt appendix before final submission if they were not saved in the repository.

13 References

References

[1] Nathan George. "All Lending Club loan data." Kaggle dataset. `https://www.kaggle.com/datasets/wordsforthewise/lending-club`.

[2] Nathan George. "EDA with Python." Kaggle notebook linked from the dataset page. `https://www.kaggle.com/wordsforthewise/eda-with-python`.

[3] Nathan George. "EDA in R: arggghh." Kaggle notebook linked from the dataset page. `https://www.kaggle.com/wordsforthewise/eda-in-r-arggghh`.

[4] Mariia Gusarova. "Detect and Remove the Outliers." Kaggle notebook. `https://www.kaggle.com/code/mariiagusarova/detect-and-remove-the-outliers`.

[5] S. Gupta, R. Gulati, and A. Chakrabarty. "Classification based credit risk analysis: The case of Lending Club." arXiv:2210.05136, 2022.

[6] B. Hadji Misheva et al. "Explainable AI in Credit Risk Management." arXiv:2103.00949, 2021.

[7] L. M. Demajo, V. Vella, and A. Dingli. "Explainable AI for Interpretable Credit Scoring." arXiv:2012.03749, 2020.

[8] S. Sanz-Guerrero and J. Arroyo. "Credit Risk Meets Large Language Models: Building a Risk Indicator from Loan Descriptions in P2P Lending." arXiv:2401.16458, 2024.

[9] M. Schwartz, Y. Wang, and S. Fang. "Enhancing ML Models Interpretability for Credit Scoring." arXiv:2509.11389, 2025.

[10] E. Nortey et al. "An Optimised Greedy-Weighted Ensemble Framework for Financial Loan Default Prediction." arXiv:2603.18927, 2026.

[11] A. Solozobov. "Label-Free Detection of Governance Evidence Degradation in Risk Decision Systems." arXiv:2604.17836, 2026.

A Appendix

Project repository: lindaperez/CreditRiskRAG. Repository artifacts support reproduction from raw data to final outputs. Place the Kaggle accepted and rejected LendingClub files under `Data/archive/`; run accepted-loan EDA, cleaning, preprocessing, model notebooks 1–9, SHAP interpretation, corpus chunking, retrieval indexing, reason selection, RAG/no-RAG generation, judging, and result aggregation in order. Main artifact folders are `EDA/accepted_eda_outputs/`, `Cleaning/cleaning_outputs/`, `Modeling/Preprocessing/preprocessing_outputs/`, `Modeling/modeling_outputs/`, `Interpretation_SHAP/shap_outputs/`, `corpus/`, `retrieval/`, and `generation/output/`.

Installation and setup: use Python 3.11. Clone the repository with `git clone https://github.com/lindaperez/CreditRiskRAG.git` and enter the project folder. For `venv`, run `python3.11 -m venv .venv`, `source .venv/bin/activate`, `python -m pip install --upgrade pip`, and `python -m pip install -r requirements.lock.txt`. For Conda, run `conda env create -f environment.yml` and `conda activate credit-risk-rag`. For notebook execution, register the kernel with `python -m ipykernel install --user --name credit-risk-rag --display-name "Python (credit-risk-rag)"`. The README also recommends running `python scripts/reproducibility_check.py`, or the faster development check with `--skip-package-check --skip-hash-check`.

Notebook inventory: accepted-loan EDA, accepted-loan cleaning, preprocessing notebook 0, model notebooks 1–9 under `Modeling/`, and SHAP interpretation notebook 1. Script inventory for letter generation: `corpus/chunk_corpus.py`, `retrieval/build_index.py`, `retrieval/hybrid_retriever.py`, `generation/select_reasons.py`, `generation/generate_letter.py`, `generation/generate_all.py`, `generation/generate_norag.py`, `generation/judge.py`, and `generation/results.py`.

B Statement of Contributions

Linda Perez Penaranda: collaborator; credit-risk framing; EDA; cleaning; modeling; policy analysis; reproducibility; demo development; final report; and report integration. Yashaswi Aryan: collaborator; model/retrieval workflow; SHAP interpretation; project development; and review of the integrated prototype. Siddharth Agarwal: collaborator; model/retrieval workflow; letter generation; project development; and review of the integrated prototype. All team members contributed as collaborators to the final project direction and implementation.